Advanced Data Management: Every Method with Practice, Solution and Gotcha

Chapter by chapter: first one to two pages of what to understand, then every method of the chapter with a practice question stated in full, its solution worked right underneath, and the trap that costs points

Advanced Data Management, chapters 1–6

2026-11-01

How to use it in three hours. For each method: read the method box, cover the green *Solution* box, do the practice on paper, uncover and compare line by line, then read the red *Gotcha*. Every number was recomputed by a script from the data as stated. Formulas that the exam's formula sheet provides are marked [formula sheet] and used, not re-derived.

Order if short on time: read the six "exactly what to know" essays first (one to two pages each), then the methods in this order: 4 (index or scan), 7 (joins), 12, 15, 13 (the plan: estimate, time, fix), 14 and 17 (EAT, flooding), 19 (tickets); then 11, 6, 8, 9, 16, and 20 as a self-test.

Concept: The database used in every practice: `bookly`, PostgreSQL 16, 8 KB pages

Table	Rows	Pages	Rows/page	Notes
`orders`	3,000,000	28,038	107	48 B of column data per row; columns `id, customer_id, product_id, order_ts, amount, status, country`; inserted in `id` and time order
`customers`	300,000	2,500	120	`customers_pkey` on `id`, fanout 250
`products`	50,000	500	100	

Indexes on `orders`: `orders_pkey` on `id` (fanout 250, 250 entries per leaf), `orders_cust_idx` on `customer_id` (3 levels, 350 entries per leaf), `orders_ts_idx` on `order_ts` (3 levels, 350 per leaf, correlation 0.98). `pg_stats`: `status` MCV `shipped` 0.80, `pending` 0.15, `returned` 0.05; `country` MCV `MA` 0.60, `FR` 0.20; `product_id` 50,000 distinct, no MCV list; `customer_id` 300,000 distinct (10 orders per customer); `order_ts` uniform over 730 days; `amount` uniform 5–205, histogram of 10 equi-depth buckets with bounds $5, 25, 45, \ldots, 205$. Prices: `seq_page_cost` 1, `random_page_cost` 4, `cpu_tuple_cost` 0.01, `cpu_index_tuple_cost` 0.005, `cpu_operator_cost` 0.0025. Devices: RAM 0.1 μs, SSD 100 μs, HDD 10 ms per random page; sequential 500 MB/s. `work_mem` 4 MB (512 pages) unless stated.

Contents

1 Which component owns the symptom **4**

2 Storage estimate and what VACUUM does **7**

3 B^+ tree height, lookup, latency **9**

4 Index or scan: pages touched, with the correlation **11**

5 Which index form **13**

6 External sort and "what changes with M" **15**

7 Join cost comparison **17**

8 The tree: blocking or streaming, and why LIMIT does not help **18**

9 Node-line arithmetic: loops, rows, time, Buffers **20**

10 Reconstruct the query from the plan **21**

11 Selectivity and estimated rows **24**

12 Where an estimate comes from, the failure, the command **25**

13 Choosing the fix: what each proposal changes in the plan **26**

14 Effective access time and hit ratio **30**

15 Where the time goes **31**

16 Replacement trace: LRU, FIFO, clock **32**

17 Sequential flooding, rings, sizing the pool **33**

18 Steal, no-force, WAL: what the log must hold **34**

19 The ticket method, with three worked tickets **35**

20 Quick checks: the letter and the one line **36**

Chapter 1: Architecture and the life of a query

Chapter 1: exactly what to know

What this chapter is about

A database management system is the layer between an application that thinks in *rows and queries* and a disk that only knows *pages and bytes*. Everything in the course follows from one fact: a page on disk costs between a thousand and a hundred thousand times more to reach than a word in memory, so the whole system is organised to decide, before touching anything, *which pages* it needs and *in what order*. The DBMS is split into subsystems, each owning one kind of decision, and the exam's first reflex is to know which one owns a given symptom.

The five subsystems

The **session manager** accepts connections and gives each one a worker; it owns nothing about data. The **query processor** turns SQL into an executable plan: it is itself a pipeline of parser, binder, rewriter, planner and executor. The **storage manager** owns the files: heap files of pages, the indexes beside them, and the **buffer pool** that keeps a subset of pages in memory. The **transaction manager** owns concurrency and durability: locks, the log, recovery after a crash. The **catalog** is a set of ordinary tables that describe the schema (which tables, which columns, which types, who may do what) and the **statistics** the planner relies on. When a question says "who caught this", the answer is the subsystem that *had the information* needed to notice: grammar lives in the parser, names and types in the catalog consulted by the binder, costs in the planner, row counts in the statistics, memory residency in the buffer pool, conflicts in the lock table.

The life of a query

A statement passes through five stages, and each produces one object. The **parser** checks the grammar only and produces a parse tree; it does not know whether a table exists. The **binder** resolves every name against the catalog, checks types and permissions, and produces a bound tree; this is where "column does not exist" and "cannot sum a text" are raised. The **rewriter** transforms the bound tree into a *logical* plan in relational algebra, applying rules that are always worth it: views are inlined, subqueries flattened, constants folded, and above all **selection pushdown** (a filter on one table is applied before that table is joined) and **projection pushdown** (columns nobody needs are dropped early). These rules need no statistics, only algebra: a selection on R commutes with a join on R and S, and joins are commutative and associative, which is exactly why the join *order* is free and someone must choose it. The **planner** (optimiser) makes the *physical* choices by estimated cost: scan or index for each table, which join algorithm, in which order, and it needs the statistics for every one of these. The **executor** runs the plan tree it is given; it chooses nothing.

The iterator model

Every node of a plan tree implements the same three calls, `open`, `next`, `close`. The root asks its child for the next row, which asks its child, down to a scan that reads a page and returns a row; rows flow up one at a time, so a Seq Scan feeding a Filter feeding a Nested Loop feeding a Limit never materialises

the whole table anywhere. This is **streaming**, and it is why a `LIMIT 20` on a streaming chain stops after twenty rows. Some nodes cannot stream because their first output depends on their last input: a Sort cannot emit its smallest row before seeing every row, a HashAggregate cannot finalise a group's count before the last row of that group, the build side of a Hash Join must be complete before the first probe. These are **pipeline breakers**. Everything above a breaker waits for it, so a LIMIT above a Sort saves only the output phase of the sort, and a slow query with a LIMIT on top is usually slow below a breaker. Breakers also hold their whole input in memory, which is where `work_mem` and spills to disk enter.

The SQL you read and the order it runs

SQL is written SELECT first but evaluated FROM first: FROM and JOIN build the rows, WHERE filters them, GROUP BY groups, HAVING filters groups, SELECT computes the output columns, ORDER BY sorts, LIMIT cuts. This explains the three classic errors: an alias defined in SELECT cannot be used in WHERE (it does not exist yet) but can in ORDER BY; a condition on an aggregate belongs in HAVING, not WHERE; every column in SELECT must be grouped or aggregated once there is a GROUP BY. Reading a plan is reading this order bottom-up.

Two workloads

OLTP is many short transactions that read or write a few whole rows by key: a web shop placing an order. It wants row storage (a row is one tuple on one page), B-tree indexes on keys, and a buffer pool that keeps the hot rows resident, so that a transaction touches a handful of cached pages. **OLAP** is few long queries that scan millions of rows but only a few columns: a report of revenue per country per day. It wants **column storage**, where each column is its own file: a query that needs three of twenty-five columns reads three files, twelve percent of the bytes, and similar values compress far better side by side. The price is symmetric: writing one row means touching every column file, and reading one whole row means reassembling it from all of them. Neither layout is better; they serve different access patterns, and the usual answer to "move everything to a column store" is *both*: rows for the application, a column copy (Parquet, DuckDB) or column-friendly indexes (BRIN, covering) for the reports.

1 Which component owns the symptom

Method

The idea. A query passes through five stages, each with one job: **parser** (grammar only) → **binder** (every name, type and permission against the catalog) → **rewriter** (algebra: views, pushdown) → **planner** (access path, join algorithm and order, by cost from statistics) → **executor** (runs the plan through the buffer pool). Beside them: the transaction manager (locks, log) and the catalog (schema and statistics).

1. Ask "what information was needed to notice the problem?" Grammar → parser. Existence of a name, a type, a right → binder. A cost or a row count → planner, and if the count is wrong, statistics. A page that was not in memory → buffer pool. Two transactions → transaction manager.
2. Name the component *and* the object it consulted (grammar, catalog, statistics, page table, lock table).

Practice

Given. Six symptoms on `bookly`: (a) `SELCT * FROM orders;` fails. (b) `WHERE custmer_id = 5` fails. (c) `SUM(status)` fails. (d) A query reads all 28,038 pages of `orders` although `orders_ts_idx` would touch 1,200. (e) The same query's `rows=` is 100 times below `actual rows`. (f) Two payroll jobs wait on each other forever.

Your task. For each: the component, what it consulted, and one phrase of why.

Solution

(a) **Parser**, the grammar: `SELCT` is not a keyword, the statement does not parse. (b) **Binder**, the catalog (`pg_attribute`): the statement is grammatical, but no column of that name exists. (c) **Binder**, the catalog types: `status` is text, `SUM` needs a number; a type error is a binding error. (d) **Planner**, the cost model: it chose the Seq Scan because its costing said so; if the choice is wrong with correct statistics, the cost constants are off (`random_page_cost` 4 on an SSD). (e) **Statistics**, `pg_stats`: a 100× estimate error is stale or correlated statistics, fixed by `ANALYZE` or `CREATE STATISTICS`, not by the planner. (f) **Transaction manager**, the lock table: a deadlock, detected by the waits-for cycle, one job is aborted.

Gotcha

"The parser rejects a misspelled column": the parser knows no columns. "The executor chose the scan": it only runs what the planner chose. A wrong estimate is a statistics problem even when the symptom is a bad plan.

Chapter 2: Storage, pages are the cost

Chapter 2: exactly what to know

Why pages

Memory is addressed by the byte in nanoseconds; a disk delivers whole blocks after a delay of microseconds (SSD) or milliseconds (HDD) that is paid *per request*, not per byte. The DBMS therefore moves data in fixed **pages** of 8 KB and counts cost in pages touched. Two consequences shape every estimate in the course. First, the number of rows is irrelevant until it is converted into pages: a thousand rows on one page cost one read, a thousand rows on a thousand pages cost a thousand. Second, the *order* of the reads matters as much as their number: a sequential read of consecutive pages streams at hundreds of megabytes per second, about sixteen microseconds per page, while a random read pays the full latency every time, a hundred microseconds on an SSD and ten milliseconds on a disk. The planner's price list says exactly this: a sequential page costs 1, a random page 4.

The memory hierarchy, in numbers you must carry

Register and cache: a nanosecond. RAM: a hundred nanoseconds. SSD: a hundred microseconds. HDD: ten milliseconds. Each step is two to five orders of magnitude, so a workload that hits RAM 99 % of the time and the disk 1 % of the time still spends almost all its time on the 1 % (chapter 6 makes this precise). Everything the storage manager does, slotted pages, indexes, buffer pools, exists to turn random disk reads into sequential ones or into memory hits.

How a row sits on a page

PostgreSQL stores a row as a **tuple**: a 24-byte header (transaction ids for visibility, a null bitmap, flags) followed by the column data, the whole thing padded to a multiple of 8 bytes. A page holds a 24-byte page header, then a **slot directory** that grows downwards from the header, one 4-byte slot per tuple giving its offset and length, while the tuples themselves grow upwards from the end of the page. The two meet in the middle; a row never straddles pages. This is the **slotted page**, and the slot is why a row's address, the `ctid`, is (page, slot) rather than a byte offset: the tuple can be moved inside its page, for instance when free space is compacted, without changing its address and therefore without touching any index that points to it. It also makes variable-length rows cheap: the slot knows where each one starts. Very large values are not stored inline at all but in a side table (TOAST), which is why a table can be smaller than its column sizes suggest. The rows-per-page computation is just this layout written down: usable bytes $8192 - 24$, divided by tuple size plus its 4-byte slot, rounded down.

Heap, sorted, hashed

A **heap file** keeps rows in no particular order, wherever there is room: inserts are one page write at the end or in a page with free space, a full scan reads every page, and a lookup by value reads half the file on average when the value exists and the whole file when it does not, since nothing tells it to stop. A **sorted file** supports binary search, $\log_2 B$ pages for a lookup, and makes range queries read only the pages of the range, but inserting in the middle shifts everything after it, up to the whole file. A **hashed file** sends each value to a bucket page by a hash function, so equality is about one page, but a range or an ordering is useless to it: consecutive values land in unrelated buckets. Real systems do not choose among these for the table itself; they keep a heap, which is cheap to grow, and add **indexes** (chapter 3) that give the sorted or hashed access on the side.

What PostgreSQL does with deleted rows

A DELETE or an UPDATE does not remove a tuple; it marks it dead so that older transactions can still see it. Dead tuples accumulate (`n_dead_tup`) and keep their space until **VACUUM** runs. VACUUM reclaims the space *inside* each page, so new rows can reuse it, and it truncates the file only at the end, when the last pages are entirely empty. After deleting 40 % of a table at random, every page still holds live rows, nothing is truncated, and the file keeps its size; the table simply has room to grow into. Only **VACUUM FULL** or **CLUSTER** rewrite the file to its live size, under an exclusive lock, and CLUSTER additionally orders the rows by an index, which makes their physical order match that key (chapter 3 calls this correlation 1). The catalog remembers the planner's view of the size, `reltuples` and `relpages`, refreshed by ANALYZE, while `pg_relation_size` reports the real file; the gap between them is dead space or TOAST.

When the disk is not the bottleneck

A query whose pages are all in memory costs CPU, not I/O: regular expressions, JSON extraction, arithmetic over millions of cached rows. Its tell-tale is a `Buffers` line with `read=0` and a second run that is no faster than the first. More cache does nothing for it; a better plan, an index that avoids visiting the rows, or less work per row does. The course's warning is the mirror image: a query that is fast the second time was served by the cache, which says nothing about its plan.

2 Storage estimate and what VACUUM does

Method

1. record = column bytes +24 (tuple header), rounded *up* to a multiple of 8.
2. rows per page $r = \lfloor (8192 - 24)/(\text{record} + 4) \rfloor$: 24 B page header, 4 B slot per row, floor because rows never straddle pages.
3. pages $B = \lceil N/r \rceil$; size = $8B$ KB. Compare with `pg_relation_size`: a larger file means dead tuples or TOAST.
4. time of a sequential scan = $8B$ KB / 500 MB/s; of p random pages = $p\times$ device latency.
5. **VACUUM** marks dead tuples' space reusable inside each page and truncates only empty *trailing* pages: after a scattered delete the file keeps its size. **VACUUM FULL** and **CLUSTER** rewrite the file to $\lceil \text{live rows}/r \rceil$ pages.

Practice

Given. `orders`: 3,000,000 rows, 48 B of column data per row.

Your task. (a) Rows per page, pages, size in MB. (b) Time of one sequential scan, and of 1,000 random page reads on SSD and on HDD. (c) 40 % of the rows are deleted at random and a normal `VACUUM` runs: pages in the file? Then `VACUUM FULL`: pages?

Solution

(a) record = 48 + 24 = 72 B (a multiple of 8 already); per row on the page 72 + 4 = 76 B; $r = \lfloor 8168/76 \rfloor = \lfloor 107.5 \rfloor = 107$; $B = \lceil 3{,}000{,}000/107 \rceil = 28{,}038$ pages; size = $28{,}038 \times 8$ KB = 219 MB.

(b) Sequential: 219 MB /500 MB/s = 0.44 s. Random: $1{,}000 \times 100\,\mu$s = 0.1 s on SSD; $1{,}000 \times 10$ ms = 10 s on HDD. One thousand random pages on an HDD cost 23 times more than reading the whole table sequentially: that is why "pages touched, and in what order" is the whole cost question.

(c) After `VACUUM`: still 28,038 pages. The dead rows are scattered over every page, so no trailing page is empty and nothing is truncated; the freed space inside pages is reused by future inserts. After `VACUUM FULL`: live rows = 1,800,000, $\lceil 1{,}800{,}000/107 \rceil = 16{,}823$ pages (131 MB), a rewrite under an exclusive lock.

Gotcha

Dividing by the column bytes alone (gives 170 rows per page). Forgetting the floor. "VACUUM shrinks the file". Counting rows where the question is pages.

Chapter 3: Indexes

Chapter 3: exactly what to know

What an index is

An index is a second, smaller structure stored beside the heap that maps a key value to the address of the row, the `ctid`. It lets the system find the pages that hold the matching rows without reading

the others, and that is the only thing it does: the rows themselves still live in the heap, and unless the index contains every column the query needs, each match ends with a visit to a heap page. Every index entry is the key plus a six-byte pointer, so an index on a four-byte integer is roughly sixteen bytes per row, a few percent of a wide table and a large fraction of a narrow one.

The B$^+$ tree

The default index is a B$^+$ tree, a balanced tree with one page per node. Inner nodes hold only *routing keys* that say which child to descend into; all the actual entries sit in the **leaves**, sorted, and the leaves are chained left to right so that once a search has reached the first matching leaf, a range is read by walking the chain without going back up. Because a page holds hundreds of keys, the tree is extremely flat: with a fanout of 250, two levels cover 62,500 entries and three cover fifteen million, so a tree over millions of rows is three or four levels deep, and the top levels are so hot that they are always in memory. A point lookup therefore costs the height in index pages plus one heap page, and in practice one or two real reads. Insertion keeps the tree balanced by **splitting**: a full leaf is cut in two, the upper half moves to a new page chained next to the old one, and the first key of the new page is pushed into the parent; if the parent is full it splits too, and a split of the root adds a level, which is the only way the tree grows taller. Deletion merges or borrows from a sibling when a page falls under half full. All of this is logarithmic in the number of entries and, because of the fanout, practically constant.

The question that decides everything: where are the matching rows?

Suppose a predicate keeps k rows. The index finds their addresses cheaply; the cost is in fetching them. If the heap happens to be in the same order as the index key, because the table is appended in time order and the key is the time stamp, or because CLUSTER rewrote it in key order, the k rows are *contiguous*: they fill k/r consecutive pages, read sequentially. If the heap is in no relation to the key, the k rows are *scattered*: each one tends to sit on its own page, each fetched by a random read, and once k exceeds a few rows per page there is hardly a page without a match, so the index scan reads the whole table in random order, worse than reading it sequentially without the index. PostgreSQL measures this relation as the **correlation** between physical position and key value, from -1 to 1, and it is the single statistic that tells you which of the two worlds you are in. The formula for the scattered case, the expected number of pages with at least one match out of B pages holding r rows each, is the course's way of turning rows into pages; the contiguous case is simply k/r. A **clustered** index is the special case where the heap is kept in key order on purpose; there can be only one, since the heap has one order; every other index is **secondary** and lives in the scattered world unless the data happens to be ordered.

Between index and scan: the bitmap

When the matches are scattered but not too many, there is a middle path. A **bitmap scan** first collects all matching row addresses from the index, sorts them by page, and then reads each needed heap page exactly once, in physical order. It never reads a page twice and never jumps backwards, so it costs about the number of distinct pages touched, at a sequential-like price, instead of one random read per row. It is what PostgreSQL chooses for predicates that keep a few percent of the rows; its tell-tale lines are `Bitmap Index Scan`, `Bitmap Heap Scan`, `Recheck Cond` and `Heap Blocks: exact=`, the last one being the count of pages it visited. The one thing a bitmap scan cannot do is avoid the pages: if the matches are on 99 % of the pages, it reads 99 % of the table.

Index forms, and what each one can answer

A **composite** index on (a, b) is sorted by a and, within equal a, by b; a search can descend on a alone, on a and b, or on a = and a range of b, but not on b alone, because b's values are scattered across the whole index. Hence the rule: equality columns first, the range column last. An **expression** index stores the value of an expression, `lower(email)`, and serves predicates on exactly that expression; an index on `email` is useless for `lower(email)` because the stored keys are different values. A **partial** index covers only rows that satisfy a predicate, `WHERE status = 'pending'`: it is small, stays small as rows leave the predicate, and is used only by queries that repeat the predicate. A **covering** index adds non-key columns with `INCLUDE`, so that a query needing only those columns is answered from the index alone, an **index-only scan** with `Heap Fetches: 0`, which removes the heap visit that was the whole cost of the scattered case. A **BRIN** index stores the minimum and maximum of a block range of pages, a few bytes per range, and lets a scan skip ranges that cannot contain the value; it is tiny and only works when the column is physically ordered, which is exactly the append-only time-stamp case. A **hash** index answers equality in one hop and nothing else. **Bitmap indexes** in the classic sense, a bit vector per distinct value, serve low-cardinality analytics with AND and OR as bit operations; PostgreSQL does not store them but builds the bitmaps on the fly, which is the bitmap scan above. GIN and GiST serve documents, text and geometry.

What an index costs

Every INSERT must add an entry to every index on the table, each one a random page write, so a table with five indexes writes six pages per insert; an UPDATE of an indexed column is a delete plus an insert in that index; and every index is another access path the planner must cost. The index that nobody uses (`idx_scan = 0`) is pure cost. Keep indexes on primary and foreign keys, on selective filters and on columns used for ORDER BY; prefer one composite index to several single-column ones; and do not index a value that most rows share, because its matches are on every page anyway.

3 B^+ tree height, lookup, latency

Method

1. Height h: the smallest h with $F^h \geq N$, bracket N between powers of the fanout [formula sheet]. Typical 3–4; never $\log_2 N$.
2. Point lookup = h index pages +1 heap page. Absent key: h. Index-only: h.
3. Range of k matches = $h + (\lceil k/L \rceil - 1)$ leaf pages + heap pages [formula sheet]: $\lceil k/r \rceil$ if the heap is in key order, k if scattered, 0 if index-only.
4. Time = reads × device latency; levels that are cached cost the RAM price. Say which target is met and what to buy.

Practice

Given. `orders_pkey`: 3,000,000 entries, fanout 250, 250 entries per leaf. `GET /orders/{id}` must answer in 1 ms.

Your task. (a) Height and page reads per lookup. (b) Latency cold on SSD and on HDD. (c) Root and second level always cached: latency on SSD. (d) A range of 10,000 consecutive ids: page reads, knowing that `orders` is stored in `id` order; and if it were not.

Solution

(a) $250^2 = 62{,}500 < 3{,}000{,}000 \le 250^3 = 15{,}625{,}000$, so $h = 3$. Lookup $= 3 + 1 = 4$ page reads.

(b) SSD: $4 \times 100\,\mu\text{s} = 0.4$ ms, the target is met. HDD: 4×10 ms $= 40$ ms, missed by $40\times$.

(c) Two hits at $0.1\,\mu$s and two reads (leaf, heap) at $100\,\mu$s: ≈ 0.2 ms. On an HDD it would still be 20 ms: the device, not the cache, decides.

(d) Index pages $= 3 + \lceil 10{,}000/250 \rceil - 1 = 3 + 40 - 1 = 42$. Heap in key order: $\lceil 10{,}000/107 \rceil = 94$ contiguous pages: total 136 reads. Scattered heap: 10,000 random pages: total 10,042, 74 times more. The heap order, not the index, decides a range's cost.

Gotcha

$\log_2 3{,}000{,}000 \approx 22$ levels: that is a binary tree. Forgetting the heap page. Charging cached levels at the device price. Counting matching rows when the heap is in key order (it is pages).

4 Index or scan: pages touched, with the correlation

Method

The idea. An index scan costs what it touches in the *heap*, and that depends on where the matching rows sit, not on how many there are.

1. s and $k = sN$ from 11.
2. Read `pg_stats.correlation` for the column. Near ± 1 (append-only, after CLUSTER): the k rows are contiguous, heap pages $= \lceil k/r \rceil \approx sB$. Near 0: scattered, expected heap pages $= B\,(1-(1-s)^r)$; compute $(1-s)^r = e^{r\ln(1-s)}$; when $sr \geq 3$ it is practically every page.
3. Index cost = descent (h random pages) + leaf pages ($\lceil k/L \rceil$, sequential) + heap pages (sequential price if contiguous, random price 4 if scattered) + per-row CPU ($0.015\,k$). Seq Scan $= B + 0.01N + 0.0025N$ per predicate [formula sheet].
4. Between the plain-index threshold and a few percent, a **bitmap scan** reads the same scattered pages once, in physical order: cheaper than a plain index scan, rarely much cheaper than a Seq Scan in I/O.
5. Add the write price (one random page per index per INSERT) and decide in one sentence that quotes both costs.

Practice

Given. The dashboard runs `SELECT country, count(*) FROM orders WHERE order_ts >= now() - interval '30 days' GROUP BY country` every minute. A colleague says "a 4 % filter can never use an index, drop `orders_ts_idx`". Separately, the returns team wants `CREATE INDEX ON orders (status)` for `WHERE status = 'returned'` (5 %).

Your task. (a) For the 30-day filter: k, heap pages, Seq Scan cost, Index Scan cost, winner and factor. (b) For `status = 'returned'` with the proposed index: heap pages touched, Index Scan cost, bitmap scan cost in pages, decision. (c) What index *would* make the returns report cheap, and how many pages would it read?

Solution

(a) $s = 30/730 = 0.0411$, $k = 123{,}288$. Correlation 0.98: contiguous, heap pages $= \lceil 123{,}288/107 \rceil = 1{,}153$. Seq Scan $= 28{,}038 + 30{,}000 + 7{,}500 = 65{,}538$. Index Scan = descent $3 \times 4 = 12$ + leaves $\lceil 123{,}288/350 \rceil = 353$ + heap 1,153 (sequential) + CPU $123{,}288 \times 0.015 = 1{,}849$: total 3,367. The index wins by $65{,}538/3{,}367 \approx 19\times$. Decision: keep it; the "4 % never" rule assumes scattered rows, and these are the last 1,153 pages of the file. A BRIN index on `order_ts` would do the same job in a few pages.

(b) $s = 0.05$, $k = 150{,}000$; `status` has no physical order, correlation ≈ 0. $(1 - 0.05)^{107} = e^{107 \times \ln 0.95} = e^{-5.49} = 0.0041$, so heap pages $= 28{,}038 \times (1 - 0.0041) = 27{,}922$: 99.6 % of the table, about 5 returns per page. Index Scan $= 12 + \lceil 150{,}000/350 \rceil (= 429) + 4 \times 27{,}922 + 150{,}000 \times 0.015 = 12 + 429 + 111{,}688 + 2{,}250 = 114{,}379$, almost double the Seq Scan. Bitmap scan: the same 27,922 pages once, in order, plus 429 leaf pages: $\approx 28{,}351$ pages against 28,038 for the Seq Scan, a draw in I/O (the planner may still prefer it in cost units because it skips the per-row CPU of non-matching rows, but it reads the whole table). Decision: the index on `status` buys nothing and costs a random write per insert; do not create it.

(c) A **covering** index, `CREATE INDEX ON orders (status) INCLUDE (customer_id, amount)` (or a partial one, `WHERE status = 'returned'`, if the report always filters on returns): an index-only scan reads $\lceil 150{,}000/350 \rceil = 429$ leaf pages and *no* heap page (`Heap Fetches: 0` once VACUUM has set the visibility map), 65 times fewer pages than the Seq Scan. The heap pages were the cost; remove the need to visit them.

Gotcha

Deciding from s alone ("5 % is small, index"). Using the scattered formula on an append-ordered column, or the contiguous one on a random column. Counting k random reads when $k > B$ (the formula caps at B). Forgetting that a bitmap scan still reads the pages.

5 Which index form

Method

Match the predicate to the index that can *descend* on it:

- B-tree on one column: =, <, BETWEEN, ORDER BY, LIKE `'abc%'` on that column.
- Composite (a, b): sorted by a then b; serves a, a AND b, $a =$ AND $b >$; not b alone. Equality columns first, the range column last.
- Expression (`lower(email)`): serves exactly `WHERE lower(email) = ...`
- Partial `WHERE status = 'pending'`: small; used only when the query repeats the predicate.
- Covering `INCLUDE (cols)`: index-only scan when every needed column is in the index.
- BRIN: min/max per block range; only a physically ordered column (append-only time stamps), tiny.
- Hash: = only. No index: a predicate keeping tens of percent, or a column written far more than read.

Practice

Given. Seven queries on `orders`: (1) `WHERE customer_id = ? AND order_ts > ?`; (2) `WHERE lower(email) = ?` on `customers`; (3) the pending-orders screen `WHERE status = 'pending' ORDER BY order_ts` (15 % of rows, read every second); (4) `SELECT customer_id, amount WHERE order_ts BETWEEN ? AND ?`; (5) `ORDER BY order_ts DESC LIMIT 10`; (6) `WHERE country = 'MA'` (60 %); (7) a 2 TB append-only log table queried by month once a week.

Your task. The index to create (or none), and why, one line each.

Solution

(1) Composite `(customer_id, order_ts)`: equality first, range last; one descent lands on the customer's rows in time order. (2) Expression index `ON customers (lower(email))`; an index on `email` cannot be searched for `lower(email)`. (3) Partial `ON orders (order_ts) WHERE status = 'pending'`: 15 % of a 3 M table is 450,000 entries, sorted by time, and the index shrinks as orders ship; a plain index on `status` would be read 99 % of the table (see 4). (4) Covering `ON orders (order_ts) INCLUDE (customer_id, amount)`: index-only range, `Heap Fetches: 0`; or `orders_ts_idx` as it is, since correlation 0.98 already makes the heap pages contiguous. (5) B-tree on `order_ts`: the planner walks the leaves backwards and stops after 10 rows, no sort. (6) None: 60 % of the rows touch every page; a Seq Scan is the right plan. (7) BRIN on the time stamp: a few hundred KB for 2 TB, prunes whole block ranges; a B-tree would be tens of GB for a weekly query.

Gotcha

Composite in the wrong order (range first). An index on `email` "for" `lower(email)`. Indexing a 60 % value. BRIN on a column with no physical order.

Chapter 4: Sorting, joins and reading a plan

Chapter 4: exactly what to know

Why sorting is a disk algorithm

A table of thirty thousand pages does not fit in a sort's memory, which is a few hundred pages, so sorting it is a sequence of passes over the disk. In **pass 0** the sort reads as many pages as it has buffers, sorts them in memory, writes them back as a sorted *run*, and repeats; it produces B/M runs, each M pages long. In each **merge pass** it opens as many runs as it has buffers minus one (the last buffer collects the output), reads them in parallel, emits the smallest element each time, and writes one longer run; the number of runs shrinks by a factor of $M - 1$ per pass until one remains. Every pass reads every page once and writes it once, so the cost is $2B$ per pass times the number of passes. Two things follow that the exam likes. The passes are a *ceiling of a logarithm*: a sort that needs two merge passes with 100 buffers needs one with 168 and still one with 10,000, so doubling the memory removes at most one pass and often none; the only sharp improvement is when the whole input fits and no merge is needed at all. And PostgreSQL tells you which world a sort was in: `Sort Method: quicksort Memory: 200kB` means it fit in `work_mem`; `external merge Disk: 230000kB` means it spilled, and the number is roughly the memory that would have avoided it, to be granted to that statement with `SET LOCAL`, not to the whole server, since `work_mem` is allocated per sort and per connection.

The four join algorithms and what each one is good at

A join combines two inputs on a condition, and the whole question is how many times each input is read. The **nested loop** takes each row of the outer input and looks for matches in the inner; done naively per row it reads the inner once per outer *row*, which is absurd for a large outer, so the real algorithm is the **block nested loop**: it fills all but two buffers with a chunk of the outer, scans the inner once per chunk, and pays the inner's size times the number of chunks. If the outer fits in memory the inner is read exactly once, and the block nested loop is as cheap as any join; it also accepts any condition at all, inequalities and LIKE included, which the other joins cannot. The **index nested loop** replaces the inner scan by an index lookup per outer row: wonderful when the outer is a handful of rows and the inner is indexed on the join key, catastrophic when the outer is large, because each lookup is a descent plus random heap reads, and a plan with `loops=120000` on its inner side is this disaster in print. The **sort-merge join** sorts both inputs on the key and merges them in one pass; the merge itself reads each input once, but the sorts cost full external sorts unless an input is already ordered, by an index or by an earlier node, and it is the natural choice when the output must be ordered by the join key anyway or when the condition is a range. The **hash join** builds a hash table on the smaller input, the **build** side, and streams the larger one through it, the **probe** side; when the build fits in memory each input is read exactly once and there is nothing cheaper for a large equi-join. When it does not fit, the **Grace** variant partitions both inputs by a hash of the key into pieces that do fit, writes them out, and joins piece by piece, reading and writing each input once and reading it again: three times the sum of the sizes, which PostgreSQL reports as `Batches` greater than one. A hash join cannot do inequalities, and a key where one value dominates defeats the partitioning. The planner keeps all four because each is the best somewhere, and `enable_hashjoin = off` does not remove one, it adds a huge penalty to its cost, for experiments only.

Aggregation and order

GROUP BY is executed either by hashing (a **HashAggregate**, one bucket per group, fine when the groups fit in memory, a pipeline breaker because a group's total is final only at the end) or by sorting the input and grouping consecutive rows (**Sort** then **GroupAggregate**, which streams group by group and is preferred when the input is already sorted or the groups are too many to hash). DISTINCT is the same choice. ORDER BY is a Sort node unless an index already delivers the order. The cheap way to produce an ordered aggregate is to aggregate first, so that the sort runs on the groups, not on the millions of input rows.

Reading a plan

`EXPLAIN` shows the planner's plan with its *estimates*: for each node a start cost (the work before the first row leaves it), a total cost in abstract units, an estimated row count and a width. `EXPLAIN ANALYZE` runs the query and adds what actually happened: the time to the first and to the last row, the actual row count, and the number of *loops*, because a node on the inner side of a nested loop runs once per outer row and all its actual figures are per loop. The tree is drawn by indentation; the first child of a join is the outer or probe side, the second the inner or build side, and execution begins at the leftmost leaf of the deepest breaker. Below each node are the lines that explain its work: a `Filter` is tested on every row after it was read, with `Rows Removed by Filter` counting the waste; an `Index Cond` was answered inside the index; a `Hash Cond` or `Merge Cond` is the join condition; `Sort Method` and `Buckets Batches Memory Usage` say whether a sort or a hash stayed in memory; `Group Key` and `Sort Key` name the GROUP BY and ORDER BY. From these lines the original SQL can be rebuilt almost word for word: scans are the FROM clause, conditions the ON and WHERE, keys the GROUP BY and ORDER BY, a Limit node the LIMIT. The habit to acquire is to read the plan bottom-up, compare estimated and actual rows at each node, and find the lowest node where they disagree, because everything above it was planned on that wrong number.

6 External sort and "what changes with M"

Method

1. runs $= \lceil B/M \rceil$ (pass 0 sorts M pages at a time). Fan-in $= M - 1$ (one buffer for the output).
2. merge passes: divide the run count by $M - 1$ with a ceiling, repeat until 1, count the divisions [formula sheet].
3. total I/O $= 2B \times (1 +$ merge passes$)$: every pass reads and writes all B pages.
4. "What changes with M'": recompute; a doubling removes at most one merge pass. One merge pass needs $\lceil B/M \rceil \leq M - 1$; no merge pass at all needs $M \geq B$.
5. PostgreSQL: `Sort Method: external merge Disk: x kB` means spilled; `work_mem` $\geq x$ makes it `quicksort Memory`. Set it with `SET LOCAL` inside the transaction.

Practice

Given. Nightly `ORDER BY amount` on `orders` (28,038 pages), $M = 100$. The DBA says "double `work_mem` and the sort will take half the time".

Your task. (a) Runs, merge passes, total I/O at $M = 100$. (b) At $M = 200$. (c) Is the DBA right? The smallest M that gives one merge pass, and the M that gives no merge pass at all, in pages and MB.

Solution

(a) Runs $= \lceil 28{,}038/100 \rceil = 281$. Fan-in 99: $281 \to \lceil 281/99 \rceil = 3 \to 1$: two merge passes. I/O $= 2 \times 28{,}038 \times (1+2) = 168{,}228$.

(b) Runs $= \lceil 28{,}038/200 \rceil = 141 \leq 199$: one merge pass. I/O $= 2 \times 28{,}038 \times 2 = 112{,}152$.

(c) From 168,228 to 112,152 is one third less, not half: passes are a ceiling of a logarithm, and the doubling removed exactly one merge pass. One merge pass needs $\lceil 28{,}038/M \rceil \leq M - 1$, i.e. $M(M-1) \geq 28{,}038$: $M = 168$ ($168 \times 167 = 28{,}056$), so 200 was already past the useful point and 400 would change nothing. No merge pass needs the whole input in memory, $M \geq 28{,}038$ pages $= 219$ MB of `work_mem`; then the sort reads once and writes nothing.

Gotcha

Fan-in M instead of $M - 1$. Forgetting pass 0 in the total. Counting reads only. "Doubling memory halves the time".

7 Join cost comparison

Method

1. Write the data: $B_R, B_S, |R|, |S|, M$, distinct values, indexes, existing order. The smaller input is the outer (nested loop) or the build (hash).
2. For each algorithm settle the "fits" case first, then its formula [formula sheet]: block NL chunks $= \lceil B_R/(M-2) \rceil$; hash in memory iff $B_{\text{build}} \leq M$, else Grace $3(B_R + B_S)$; sort-merge pays the *full* external sort (6) of each unsorted input, then $B_R + B_S$; index NL $= B_R + |R| \times (h +$ matching heap pages).
3. Pick the minimum. Name the trap answers (rows where pages belong; M for $M-2$).
4. Say when the loser wins: more memory (both read each input once), an index with a tiny outer, sorted output wanted.
5. ORDER BY on the result: aggregate first, then sort the groups, never the big input.

Practice

Given. Payroll joins `customers` (R: 2,500 pages, 300,000 rows) with `orders` (S: 28,038 pages) on `customer_id`, sums `amount` per customer, ordered by `customer_id`. `orders_cust_idx` exists (3 levels); each customer has 10 orders, scattered. $M = 50$, then $M = 3{,}000$.

Your task. (a) Block nested loop. (b) Hash join. (c) Sort-merge join. (d) Index nested loop with `customers` outer. (e) Redo (a)–(c) at $M = 3{,}000$, decide, and say how to get the sorted output.

Solution

(a) $M = 50$, block NL. Chunks $= \lceil 2{,}500/48 \rceil = 53$; cost $= 2{,}500 + 53 \times 28{,}038 = 1{,}488{,}514$ page I/Os: `orders` read 53 times.

(b) Hash. Build $=$ `customers`, $2{,}500 > 50$: Grace, $3 \times (2{,}500 + 28{,}038) = 91{,}614$. Sixteen times cheaper than the block NL.

(c) Sort-merge. `orders`: runs $\lceil 28{,}038/50 \rceil = 561$, fan-in 49: $561 \to 12 \to 1$, two merge passes, $2 \times 28{,}038 \times 3 = 168{,}228$. `customers`: runs 50, $50 \to 2 \to 1$, two merge passes, $2 \times 2{,}500 \times 3 = 15{,}000$. Merge 30,538. Total 213,766.

(d) Index NL. $2{,}500 + 300{,}000 \times (3 + 10) = 3{,}902{,}500$: every customer costs a descent plus 10 random heap pages; the worst of all, because the outer is not tiny.

(e) $M = 3{,}000$. Block NL: $\lceil 2{,}500/2{,}998 \rceil = 1$ chunk, 30,538. Hash: build fits, 30,538. Sort-merge: `orders` runs $\lceil 28{,}038/3{,}000 \rceil = 10$, one merge pass, 112,152; `customers` fits in memory, 2,500; merge 30,538; total 145,190. Decision: the hash join with `customers` as build (2,500 pages $= 20$ MB of `work_mem`); block NL ties in I/O but does $300{,}000 \times 3{,}000{,}000$ comparisons in memory. Sorted output: HashAggregate the join (300,000 groups, a few tens of MB) and sort the 300,000 result rows, not the 28,038-page input; or, if memory is tight, the sort-merge join delivers the result already in `customer_id` order, which is the one case where its 145,190 buys something.

Gotcha

$|R| = 300{,}000$ in the block NL formula (that is the simple nested loop, 8×10^9). M instead of $M - 2$. A sort-merge without the sort cost. Sorting the big input for ORDER BY.

The plan used by the plan-reading methods of chapters 4, 5 and 6

The plan below is used by 8, 9 and 10 (chapter 4), 12 and 13 (chapter 5) and 15 (chapter 6). It is the real `EXPLAIN (ANALYZE, BUFFERS)` of the returns dashboard of `bookly`, `work_mem` 4 MB, parallelism off:

```
SELECT c.name, count(*) AS n, sum(o.amount) AS refunded
FROM orders o JOIN customers c ON c.id = o.customer_id
WHERE o.status = 'returned' AND o.country = 'FR'
GROUP BY c.name ORDER BY refunded DESC LIMIT 20;
```

```
Limit  (cost=115982.1..115982.2 rows=20)
       (actual time=3480.2..3480.3 rows=20 loops=1)
  Buffers: shared hit=458038 read=50000
  ->  Sort  (cost=115982.1..116057.1 rows=30000)
            (actual time=3480.1..3480.2 rows=20 loops=1)
        Sort Key: (sum(o.amount)) DESC
        Sort Method: top-N heapsort  Memory: 27kB
        ->  HashAggregate  (cost=114632.1..114932.1 rows=30000)
                  (actual time=3440.6..3462.3 rows=98210 loops=1)
              Group Key: c.name
              ->  Nested Loop  (cost=0.42..114482.1 rows=30000)
                        (actual time=0.3..3390.8 rows=120000 loops=1)
                    ->  Seq Scan on orders o
                          (cost=0.00..65538.0 rows=30000)
                          (actual time=0.1..410.5 rows=120000 loops=1)
                          Filter: ((status = 'returned') AND (country = 'FR'))
                          Rows Removed by Filter: 2880000
                          Buffers: shared hit=28038
                    ->  Index Scan using customers_pkey on customers c
                          (cost=0.42..1.63 rows=1)
                          (actual time=0.020..0.024 rows=1 loops=120000)
                          Index Cond: (id = o.customer_id)
                          Buffers: shared hit=430000 read=50000
Planning Time: 0.3 ms
Execution Time: 3482.0 ms
```

8 The tree: blocking or streaming, and why LIMIT does not help

Method

1. Indentation gives the children. The first child of a join is the outer (or probe) side, the second the inner (or build) side. Draw it top-down, arrows from child to parent.
2. Tag each node. **Blocking** (all input before the first output): Sort, Hash build, HashAggregate, Materialize, Bitmap Index Scan. **Streaming**: Seq/Index/Bitmap Heap Scan, Filter, Limit, Nested Loop, Merge Join, hash probe, GroupAggregate on sorted input. One phrase of why per node.
3. Consequence: nothing above a breaker starts before it finishes; a LIMIT above a breaker saves only the breaker's output phase. Execution starts at the leftmost leaf of the deepest breaker.

Practice

Given. The plan above.

Your task. Draw the tree (six nodes); tag each blocking or streaming with a phrase; say why `LIMIT 20` leaves the query at 3.5 s, reading it off the `actual time` values.

Solution

Limit ← Sort ← HashAggregate ← Nested Loop {outer: Seq Scan `orders`; inner: Index Scan `customers`, once per outer row}.

Node	Blocking?	Why
Seq Scan orders	no	tests the filter row by row and passes survivors on
Index Scan customers	no	one lookup per outer row, the match is emitted at once
Nested Loop	no	emits each (order, customer) pair as the inner returns it
HashAggregate	**yes**	a customer's sum is final only after its last order
Sort	**yes**	the largest sum is known only when every group is in
Limit	no	forwards the first 20 rows it receives, then stops

The join ends at 3,390.8 ms; the HashAggregate's first output is at 3,440.6 ms (it waited for the whole join); the Sort's first row is at 3,480.1 ms; the Limit gets its 20 rows at 3,480.3 ms. The LIMIT saved only the sort's output (hence `top-N heapsort` in 27 kB); the 3.4 s were spent below two breakers that cannot stop early.

Gotcha

Calling the Nested Loop blocking (it streams; its *inner lookups* are what is slow). Calling the Limit the slow node because it is on top. A Bitmap Heap Scan streams; a Bitmap Index Scan blocks.

9 Node-line arithmetic: loops, rows, time, Buffers

Method

1. `cost=start..total rows=est`: the planner's belief. `actual time=first..last rows=act loops=n`: measured, *per loop*. Totals: rows = act × loops, time = last × loops.
2. q-error of a node = max(est/act, act/est) on the *total* rows.
3. `Buffers: shared hit=h read=r`: pages requested, children included; hit ratio = $h/(h+r)$; `read` means not in `shared_buffers` (OS cache or disk). `temp read/written` means a sort or hash spilled.
4. Pages per inner lookup = $h_{\text{tree}} + 1$; expected total = loops × that; compare with hit + read.

Practice

Given. The inner Index Scan of the plan: `rows=1 loops=120000`, `actual time=0.020..0.024`, `Buffers: shared hit=430000 read=50000`; `customers_pkey` has 3 levels.

Your task. (a) Rows and time the inner side produced in total. (b) q-error of the Seq Scan and of the Nested Loop. (c) Expected pages for the inner side, and whether `hit + read` agrees. (d) Hit ratio of the inner side and of the whole query; is either number useful?

Solution

(a) Rows = 1 × 120,000 = 120,000; time = 0.024 ms ×120,000 = 2,880 ms: the inner lookups are 2.9 of the 3.5 seconds.

(b) Seq Scan: est 30,000, act 120,000, q-error = 4, an under-estimate. Nested Loop: est 30,000, act 120,000, q-error 4: inherited unchanged from its outer child (one match per order).

(c) 120,000 × (3 + 1) = 480,000 pages; `hit + read` = 430,000 + 50,000 = 480,000. Exactly: three index pages and one heap page per lookup. The index pages are hot (hit); 50,000 of the heap page fetches missed.

(d) Inner: 430,000/480,000 = 89.6 %. Whole query: 458,038/508,038 = 90.2 %. Neither tells the story: the cost is the 50,000 random `read`s on one node. Read `Buffers` per node and look at `read`, not at a ratio.

Gotcha

Reading `rows=1` on the inner side as "one row" (it is per loop). Reading `actual time=a..b` as a duration (first row, last row). "`read` means disk". Judging by the global hit ratio.

10 Reconstruct the query from the plan

Method

`Seq Scan on t a` = `FROM t a`. `Hash Cond` / `Merge Cond` / `Join Filter` = `ON`. `Filter`, `Index Cond`, `Recheck Cond` = `WHERE` (on that table). `Group Key` = `GROUP BY`; an `Aggregate` with no key = a single `count(*)` or `sum`. `Sort Key` = `ORDER BY`, an aggregate in parentheses is a computed column. `Limit` = `LIMIT`. The outer child of a Hash Join is the probe side, the `Hash` child the build side.

Practice

Given.

```
Sort
  Sort Key: c.country, (count(*)) DESC
  ->  HashAggregate
        Group Key: c.country
        ->  Hash Join
              Hash Cond: (o.customer_id = c.id)
              ->  Bitmap Heap Scan on orders o
                    Recheck Cond: (order_ts >= '2026-10-01')
                    Filter: (amount > 100)
                    ->  Bitmap Index Scan on orders_ts_idx
                          Index Cond: (order_ts >= '2026-10-01')
              ->  Hash
                    ->  Seq Scan on customers c
                          Filter: (country <> 'MA')
```

Your task. Write the SQL. Which table is the build side, and why did the planner pick it?

Solution

```sql
SELECT c.country, count(*)
FROM orders o JOIN customers c ON o.customer_id = c.id
WHERE o.order_ts >= '2026-10-01' AND o.amount > 100 AND c.country <> 'MA'
GROUP BY c.country
ORDER BY c.country, count(*) DESC;
```

Build side: `customers` (under `Hash`), filtered to 40 % = 120,000 rows, a few MB; the filtered `orders` side is probed. The planner builds on the smaller *filtered* input.

Gotcha

Putting a `Filter` of one table into the `ON` clause. Missing the second `Sort Key`. Reading the `Hash` child as the probe.

Chapter 5: Query optimisation

Chapter 5: exactly what to know

The planner's job, and why it is hard

Given a logical plan, the planner must choose for every table whether to scan it or use which index, for every join which algorithm, and in which order to join the tables; the number of combinations is enormous and the right choice depends on sizes it cannot know exactly, since the query has not run yet. It solves this with two tools: a **cost model** that assigns a number to any plan, and **statistics** that let it estimate the sizes the cost model needs. The search itself is a solved problem; the hard part is estimation, because every later decision, the algorithm, the order, the memory, the index, is a function of row counts, and a wrong count at a leaf is multiplied through every node above it.

What the statistics are

`ANALYZE` reads a random sample of about thirty thousand rows and stores, per table, the row and page counts (`reltuples`, `relpages`), and per column: the fraction of nulls; the number of distinct values (`n_distinct`, negative when it means "a fraction of the rows", for keys); the **most common values** with their frequencies, a short list that captures skew exactly; a **histogram** of the remaining values with equal-depth buckets, so that each bucket holds the same share of rows; and the **correlation** between physical order and value order. These numbers are what `pg_stats` shows and what every estimate in the course is read from. They go stale when the table changes without a new ANALYZE, and they are per column: nothing in them says how two columns relate.

From a predicate to a row count

A predicate's **selectivity** is the fraction of rows it keeps, and the estimated row count is that fraction times the table size. For an equality the planner first looks in the most-common-values list, where the frequency is exact; a value not in the list gets an even share of what is left; a column with no list gets one over the number of distinct values. For a range it counts how many histogram buckets lie on the right side of the bound, plus a linear share of the bucket that contains it, over the total number of buckets. Several predicates on one table are combined under **independence**: AND multiplies the selectivities, OR adds them and subtracts the product, NOT takes the complement. A join's size is estimated as the product of the input sizes over the larger number of distinct join values, which for a key–foreign-key join collapses to the size of the foreign-key side after its own filters. Every one of these is a reasonable default and every one has a failure mode: uniformity fails on skew, independence fails on correlated columns, and both fail silently.

The cost model

A plan's cost is the sum of its nodes' costs, each computed from the estimated input sizes and a short price list: a page read sequentially costs 1, a page read at random 4, handling a row 0.01, an index entry 0.005, an operator or hash 0.0025. The units are abstract, calibrated to sequential page reads, and they are not milliseconds; a plan of cost 65,000 is not 65 seconds. The point of the list is the ratios: random is four times sequential (the right ratio for disks, too high for SSDs, hence

`random_page_cost = 1.1` on flash), and rows are cheap compared with pages. The cost of a node is the formula for its algorithm (chapter 4) filled with the estimated numbers, and because the numbers are estimates, the planner's choice is only as good as the row counts. The `cost=start..total` on a plan line distinguishes the work before the first row from the whole; a breaker's start cost is essentially its total.

Choosing a join order

With n tables there are $n!$ ways to join them left to right, and more if the tree may be bushy; the planner does not enumerate them. **System-R dynamic programming** computes the cheapest way to produce every subset of the tables, starting with single tables, then pairs built from singles, then triples built from the best pairs, and so on; a subset's best plan is built from best plans of its parts, so only $2^n - 1$ subsets are ever costed, and for each subset the planner also keeps the cheapest plan that delivers an *interesting order*, a sort order a later merge join or the final ORDER BY could reuse. Beyond a dozen tables even this is too much, and PostgreSQL falls back to heuristics (`join_collapse_limit` stops reordering explicit joins beyond eight tables, `geqo_threshold` switches to a genetic search beyond twelve). The order matters because joining the most selective, most filtered inputs first keeps every intermediate result small; the order a developer wrote the FROM clause in reflects the business question, not the filtered sizes, and freezing it server-wide trades a planning problem on one query for bad plans on all.

When the planner is wrong, and how to tell which way

The planner is wrong in exactly three ways, and `EXPLAIN ANALYZE` exposes them by printing estimated and actual rows side by side; the ratio between them, the **q-error**, says how wrong, and the *lowest* node with a large q-error is the cause. **Stale statistics**: the table grew or changed and nobody ran ANALYZE; the tell-tale is a live tuple count far from `reltuples` or an old `last_analyze`, and the fix is `ANALYZE`. **Correlated columns**: two predicates whose columns determine each other, zone and city, status and country; each single-column frequency is right, the product is wrong, and no amount of ANALYZE helps because the statistics are per column; the fix is **extended statistics**, `CREATE STATISTICS (dependencies) ON a, b`, which measure how much one column implies the other. **Skew**: a column where a few values dominate and the rare ones are estimated at an even share; the fix is a longer most-common-values list, `SET STATISTICS 1000`. The direction of the error matters: an under-estimate makes the planner pick per-row plans, nested loops and index lookups, for an input that turns out to be large, which is the typical disaster; an over-estimate makes it pick a hash or a scan where an index would have done, which is merely slow. Rule-based optimisers, which apply fixed priorities with no data, never make these mistakes and never make the good choices either; a cost-based optimiser is wrong exactly when its statistics are, which is why the levers, in order, are statistics, then cost constants, then memory, then the `enable_*` switches for experiments, then rewriting the query, and hints last.

11 Selectivity and estimated rows

Method

The rules are on the formula sheet [formula sheet]; the method is *where to read each number*.

1. Equality: is the value in `most_common_vals`? Yes: its frequency. No: $(1 - \text{null_frac} - \sum \text{MCV freqs})/(n_{\text{distinct}} - \#\text{MCV})$; no MCV list at all: $1/n_{\text{distinct}}$.
2. Range: count whole histogram buckets on the right side of the bound, add the linear share of the bucket that contains it, divide by the number of buckets.
3. Combine under independence: AND multiplies, OR adds minus the product, NOT complements.
4. rows $= sN$. Join: $|R||S|/\max(d_R, d_S)$; a key–foreign-key join returns the foreign-key side after its filters.
5. Then say whether independence is believable (correlated columns make the AND an underestimate).

Practice

Given. The `bookly` statistics.

Your task. Estimated rows for: (a) `status = 'returned' AND country = 'FR'`; (b) `amount > 165`; (c) `status = 'returned' OR country = 'FR'`; (d) `product_id = 777`; (e) `orders JOIN customers ON customer_id = id`; (f) `order_ts >= now() - interval '30 days'`. (g) Which of (a)–(f) would you distrust, and why?

Solution

(a) Both in the MCV lists: $0.05 \times 0.20 = 0.01$; $0.01 \times 3{,}000{,}000 = 30{,}000$ rows.
(b) Bounds $5, 25, \ldots, 205$: bucket boundaries every 20; 165 is the boundary between buckets 8 and 9, so two whole buckets lie above it: $2/10 = 0.2$; 600,000 rows.
(c) $0.05 + 0.20 - 0.05 \times 0.20 = 0.24$; 720,000 rows.
(d) No MCV list: $1/50{,}000$; $3{,}000{,}000/50{,}000 = 60$ rows.
(e) $|R||S|/\max(d) = 3{,}000{,}000 \times 300{,}000/300{,}000 = 3{,}000{,}000$: every order has one customer, the join returns the foreign-key side.
(f) Uniform over 730 days: $s = 30/730 = 0.0411$; 123,288 rows.
(g) (a): returns and country are plausibly correlated (a country with a different returns policy); the product is then an under-estimate, and the fix is `CREATE STATISTICS (dependencies) ON status, country`. (d) is also fragile: a uniform guess on a column that surely has best-sellers; a longer MCV list (`SET STATISTICS 1000`) fixes it.

Gotcha

Adding for AND (0.25). Using $1/d$ for a value that is in the MCV list. Counting the bucket that contains the bound as whole. $\min(d)$ or $d_R + d_S$ in the join size.

12 Where an estimate comes from, the failure, the command

Method

1. Rebuild the planner's arithmetic: scan `rows=` = (MCV frequency, or $1/d$, or histogram share) $\times N$; each extra predicate multiplies by its own selectivity (independence); join `rows=` by the join formula.
2. Compare with `actual rows`: q-error. Find the *lowest* node where it is large: that is the cause; everything above inherits it.
3. Name the failure and its fix: **stale statistics** (`n_live_tup` $\neq$ `reltuples`, old `last_analyze`) $\rightarrow$ `ANALYZE t`; **correlated columns** (single frequencies right, product wrong; ANALYZE changes nothing) $\rightarrow$ `CREATE STATISTICS s (dependencies) ON a, b FROM t; ANALYZE t`; **skew** (a rare value estimated at $1/d$) $\rightarrow$ `ALTER TABLE t ALTER COLUMN c SET STATISTICS 1000; ANALYZE t`.
4. Say what estimate the fix produces and what the planner will do with it.

Practice

Given. The Seq Scan line: `rows=30000`, actual 120,000; `pg_stats`: `returned` 0.05, `FR` 0.20, both fresh (`last_analyze` ten minutes ago).

Your task. (a) Where does 30,000 come from? (b) q-error and direction, and why that direction is dangerous here. (c) The failure, whether `ANALYZE orders` helps, the command, the new estimate.

Solution

(a) $0.05 \times 0.20 = 0.01$; $0.01 \times 3{,}000{,}000 = 30{,}000$: two MCV frequencies multiplied under independence.

(b) $120{,}000/30{,}000 = 4$, an *under*-estimate. With 30,000 outer rows the planner priced the inner lookups at $30{,}000 \times 1.63 \approx 49{,}000$ units and chose the nested loop; with the true 120,000 rows they cost four times more, and the random heap reads behind them are the 2.9 s. Under-estimates buy per-row plans for inputs that are not small.

(c) Correlated columns: French orders are returned four times more often than average, so `status` and `country` are not independent. `ANALYZE` does not help: both single-column frequencies are already right; the product is the error. Fix:

```
CREATE STATISTICS s_ret (dependencies) ON status, country FROM orders;
ANALYZE orders;
```

The planner then scales the product by the measured dependency and estimates about 120,000, and at 120,000 it switches to a hash join (13).

Gotcha

"Run ANALYZE" for a correlation error. Fixing the top node. Adding an index for an estimate error (symptom, not cause). Forgetting the `ANALYZE` after `CREATE STATISTICS` (the object exists, the numbers do not).

13 Choosing the fix: what each proposal changes in the plan

Method

For every proposal, say what changes in four places: the **plan lines**, the **pages** (`Heap Blocks`, `Buffers`), the **estimate**, the **time**. Then order the proposals by what they cost to apply.

Proposal	What it changes
extended statistics	estimate right → join algorithm and order change; nothing else; two commands, no lock
`ANALYZE`	only stale counts
composite index on the filter columns	`Filter` moves into `Index Cond`; pages unchanged if the rows are scattered; estimate unchanged; a slower insert
CLUSTER on an index	correlation → 1: `Heap Blocks` → $\lceil k/r \rceil$, the scan collapses; the join is untouched; a one-off rewrite under lock, decays with new rows
covering / partial index	index-only scan, `Heap Fetches: 0`: no heap pages at all for that predicate
`work_mem`	`Disk`, `Batches` disappear; nothing about the plan shape
hash join alternative	build on the smaller *filtered* input (check it fits: rows × width vs `work_mem`), probe with a Seq Scan: sequential pages instead of loops of random ones

Practice

Given. Four proposals for the 3.5 s dashboard: (i) `CREATE INDEX ON orders (status, country)`; (ii) `CLUSTER orders USING` that index; (iii) the extended statistics of 12; (iv) `CREATE INDEX ON orders (status, country) INCLUDE (customer_id, amount)`.

Your task. For each: plan lines, pages, estimate, time. Then: with (iii), what plan does the planner choose, does its build side fit in 4 MB, what does it cost in units and seconds against the 3.5 s? Which proposal first?

Solution

(i) The Seq Scan becomes a Bitmap Heap Scan with `Recheck Cond` on both columns; `Heap Blocks` $= 28{,}038 \times (1 - 0.99^{107}) \approx 28{,}038 \times 0.66 = 18{,}400$ (1 % of rows, scattered): two thirds of the table, read in order; estimate still 30,000 (same independence rule inside the index condition); the 120,000 lookups remain: 3.5 s becomes about 3.3 s. Plus a random write per insert.

(ii) Rewrites 219 MB under an exclusive lock; the 120,000 matching rows become $\lceil 120{,}000/107 \rceil = 1{,}122$ contiguous pages, the scan drops to milliseconds; estimate unchanged; the join unchanged: about 3.0 s. New orders append out of order.

(iii) Estimate 120,000; the planner abandons the nested loop. Nothing else changes in the plan lines of the scan.

(iv) Index-only scan of $\lceil 120{,}000/350 \rceil = 343$ leaf pages, `Heap Fetches: 0`; the scan drops from 410 ms to a few ms; the estimate and the join unchanged: about 3.0 s.

With (iii). Hash join. Build side = the smaller input after filtering = the 120,000 `orders` rows × about 80 B = 9.6 MB > 4 MB: `Batches: 4`, so either accept the batches or `SET LOCAL work_mem = '16MB'` for `Batches: 1`. Probe = Seq Scan of `customers`: $2{,}500 + 300{,}000 \times 0.01 = 5{,}500$ units. Total $\approx 65{,}538$ (scan of `orders`) $+5{,}500 + 0.0025 \times 420{,}000 \approx 72{,}000$ units against the nested loop's $65{,}538 + 120{,}000 \times 1.63 \approx 261{,}000$ at the true row count. Time: 0.41 s for the scan + 2,500 sequential pages (20 MB, 40 ms) + hashing ≈ 0.5 s instead of 3.5 s.

Order. (iii) first: two commands, no rewrite, fixes the node that owns the time. Then (iv) if the 0.4 s scan matters, since it also makes the index the whole access path. (i) and (ii) spend effort on the 12 % of the time that the scan owns.

Gotcha

Building the hash on `customers` "because it is the smaller table": after the filter, `orders` is the smaller input. Forgetting to check the build against `work_mem`. Proposing CLUSTER for a join problem.

Chapter 6: Buffer management

Chapter 6: exactly what to know

Why the DBMS manages its own memory

The operating system already caches disk pages; the DBMS keeps its own **buffer pool** anyway, because it knows things the OS cannot: that a sequential scan will use each page once and should not evict anything, that a log record must reach disk before the data page it describes, that a page is *pinned* by an operator that is reading it right now and must not move, and how much memory it is allowed to use at all. The pool is an array of **frames** of one page each, plus a page table that maps a page identifier to the frame holding it. Every page access goes through it: a **hit** returns the frame; a **miss** chooses a victim frame, writes the victim back if its **dirty bit** says it was modified, reads the requested page into the frame, and pins it for the caller. The first access to any page is a miss by nature, a *compulsory* miss; the policy decides only what to evict after that.

What a hit ratio is worth

The effective access time is the average over hits and misses, $h\,T_{\text{hit}} + (1-h)\,T_{\text{miss}}$, and because a miss costs a thousand times a hit, the miss term dominates until the hit ratio is extremely close to one. At 98 % on an SSD the average access is twenty times slower than memory; at 99.9 % it is twice as slow; the difference between "98" and "99.9" is a factor of ten in time, which is why OLTP systems chase the last tenth of a percent. The ratio is also a global average that can hide a local disaster: a query that hits 458,000 times and misses 50,000 times has a respectable 90 %, but if the 50,000 misses are random reads on one node they are the whole three seconds. PostgreSQL reports hits and misses per node in the `Buffers` line of `EXPLAIN (ANALYZE, BUFFERS)`, and it is the `read` count on a node, not a ratio, that locates the time. A `read` means "not in `shared_buffers`": the page may still have come from the operating system's cache, which is why a cold and a warm run can differ less than expected, and why the per-miss cost in practice is between the RAM and the device figures.

Replacement policies

When the pool is full, something must go. The best possible choice is the page whose next use is furthest in the future, **OPT**, which no one can implement because it needs the future; it is the yardstick. **FIFO** evicts the page loaded longest ago, ignoring use; it is cheap and it has Belady's anomaly, where giving it more frames can increase the misses. **LRU** evicts the page used longest ago, on the assumption that recent use predicts future use; it has the *stack property*, more frames never hurt, but maintaining an exact recency order means moving a frame to the front of a list on *every hit*, under a lock shared by all the processes using the pool, and at millions of hits per second that lock becomes the bottleneck. **Clock** approximates LRU without the list: one reference bit per frame and a hand that sweeps around; a hit merely sets the bit, and a victim is found by advancing the hand, clearing bits that are set, until a frame whose bit is already clear, one that has not been touched since the hand last passed. PostgreSQL's **clock-sweep** replaces the bit by a `usage_count` from zero to five, incremented on each hit and decremented by each pass of the hand, so that a page used often survives several sweeps; it adds frequency to recency at the price of one counter. **MRU**, evicting the most recently used, is the right policy for one pathological pattern, a loop over a file larger than the pool, where LRU evicts each page just before it is needed again.

Sequential flooding, and PostgreSQL's answer

Every recency policy fails on the same workload: a sequential scan of a table larger than the pool. The scan touches each page once and never again, yet to LRU each scanned page is the most recent, so the scan marches through every frame, evicting the entire hot **working set** that the transactions were using, and gains nothing itself, since none of its pages will be reused. The transactions that follow pay one miss per hot page to reload it, the "slow morning after the nightly job". PostgreSQL does not fix this by a cleverer policy but by *intent*: a sequential scan of a table larger than a quarter of `shared_buffers` is given a private **ring** of 32 frames (256 KB) and recycles them among itself, so the scan still reads every page but evicts only its own previous pages; the working set survives if it fits in the frames outside the ring. Bulk loads and VACUUM get rings of their own; index scans never do. A table smaller than the threshold simply becomes resident after one scan. The threshold is a computation the exam asks for: frames are `shared_buffers` divided by 8 KB, the ring applies above a quarter of that.

Sizing the pool

The hit ratio climbs steeply as the pool grows towards the size of the working set, the pages the workload actually revisits, and then flattens; beyond that point more memory buys nothing, and a workload that streams, a scan or an export, sits at a hit ratio near zero whatever the size. So the pool is sized for the working set, measured with `pg_buffercache`, not for the database. The usual setting is a quarter of RAM, because the rest has other uses: the operating system's cache, which serves the `read`s cheaply, and `work_mem`, which every sort and hash of every connection takes from the same memory. An oversized pool starves both and makes checkpoints, when dirty pages are flushed, longer and burstier.

Dirty pages, commit, and the log

A modified page in the pool is dirty and must reach disk eventually. Two policy questions define a system's durability design. May a dirty page of a transaction that has not yet committed be written to disk, because the pool needs the frame? Allowing it is **steal**, and it means the system must be able to *undo* that write after a crash, so the log must hold the old value, the before-image. Must all of a transaction's dirty pages be written at commit? Requiring it is **force**, and it makes commit a burst of random page writes; **no-force** lets commit return before the pages are written, which means the system must be able to *redo* the committed change after a crash, so the log must hold the new value, the after-image. Real systems choose steal and no-force for performance, and make both safe with **write-ahead logging**: a log record is written before the data page it describes, and at commit the log alone is forced to disk, one sequential append. After a crash, recovery replays the log: committed changes are redone, uncommitted ones undone. A committed row is therefore safe in the log before it is in the data file, and "commit wrote my data" is true of the log, not of the pages.

14 Effective access time and hit ratio

Method

EAT $= h\,T_{\text{hit}} + (1-h)\,T_{\text{miss}}$. Then two ratios: EAT$/T_{\text{hit}}$ ("× RAM") and EAT$_1$/EAT$_2$ ("× the other scenario"). $h = \text{hit}/(\text{hit} + \text{read})$ from `Buffers` or `pg_statio_user_tables`. The miss term dominates until h is very close to 1: OLTP wants 99.9 %+.

Practice

Given. $T_{\text{hit}} = 0.1\,\mu$s, $T_{\text{miss}} = 100\,\mu$s (SSD). The monitoring page says "hit ratio 98 %, the pool is fine".

Your task. (a) EAT at 98 %; how many times RAM? (b) EAT at 99.9 %, and the factor between the two. (c) The dashboard's `Buffers: shared hit=458038 read=50000`: its hit ratio, and the sentence that explains why 90 % hides a 3-second problem.

Solution

(a) $0.98 \times 0.1 + 0.02 \times 100 = 0.098 + 2 = 2.1\,\mu$s: 21 times RAM. The 2 % of misses cost twenty times more than all the hits.

(b) $0.999 \times 0.1 + 0.001 \times 100 = 0.0999 + 0.1 = 0.2\,\mu$s: twice RAM, and 10.5 times faster than at 98 %. From 98 to 99.9 is "a bit more cache"; in time it is a factor of ten.

(c) 458,038/508,038 = 90.2 %. The ratio averages 458,038 cheap hits with 50,000 random reads that cost $50{,}000 \times 60\,\mu\text{s} = 3$ s; the reads are concentrated on one node and that node is the query. A hit ratio is a global average; the cost is a per-node count of `read`.

Gotcha

"98 % is good" without the EAT. Using the hit ratio of the query to judge one node. Forgetting that a miss on HDD is 10 ms, making 98 % a 200μs EAT, 2,000 times RAM.

15 Where the time goes

Method

1. Per node, compute the span: `last` × `loops`; subtract the children to get the node's own time. The node with the biggest own time owns the seconds.
2. Convert that node's work into pages: scan = pages of the table (or `Heap Blocks` for a bitmap scan, from the scattered-pages formula); inner side = loops $\times(h+1)$, split into `hit` and `read`.
3. seconds $\approx$ `read` × random price (50–100 μs SSD, 10 ms HDD) + `hit` $\times 0.1\,\mu$s + CPU per row. Check against the measured time.
4. Say in one sentence what the fix must change: the number of random reads, the number of loops, or the number of pages scanned.

Practice

Given. The plan. `customers` is 2,500 pages (20 MB); `shared_buffers` is 128 MB but busy with `orders`; a random SSD read costs 60 μs here.

Your task. (a) Own time of the Seq Scan, the inner lookups, the HashAggregate and the Sort. (b) Explain `hit=430000 read=50000` and estimate the seconds from the reads. (c) If the planner had used a bitmap scan on an index on `status`, how many `Heap Blocks` would it have shown, and would it have saved time? (d) The one sentence: what must the fix change?

Solution

(a) Seq Scan: 410.5 ms. Inner lookups: $0.024 \times 120{,}000 = 2{,}880$ ms. Nested Loop's own overhead: $3{,}390.8 - 410.5 - 2{,}880 \approx 100$ ms. HashAggregate: $3{,}462.3 - 3{,}390.8 = 72$ ms. Sort: 18 ms. The inner lookups own 83 % of the time.

(b) 120,000 lookups × 4 pages = 480,000. The three index pages per lookup are the same hot pages every time: hits. The heap page differs per customer: 98,210 distinct customers spread over 2,500 pages, but `customers`' pages keep getting evicted by `orders` traffic, so 50,000 of the fetches missed. $50{,}000 \times 60\,\mu\text{s} = 3.0$ s, which matches the 2.9 s measured: the time is the random heap reads of `customers`.

(c) `status = 'returned'` is 5 % of rows with no physical order: $28{,}038 \times (1 - 0.95^{107}) = 27{,}922$ heap blocks, 99.6 % of the table. The bitmap scan would read what the Seq Scan reads, in the same order: no time saved, and the 2.9 s of the join untouched.

(d) The fix must remove the 120,000 random lookups, not speed up the scan: get the estimate right so that the planner joins by hashing (13), or make `customers` resident.

Gotcha

Fixing the scan (indexes, CLUSTER) when the join owns the time. Summing `actual time` of nested nodes twice (children are included in the parent's span). Costing a `read` as a disk seek when it was the OS cache (hence 60, not 100, μs).

16 Replacement trace: LRU, FIFO, clock

Method

Draw the frames; process the reference string one request at a time; count misses (and writes if dirty).

- **LRU**: hit moves the page to the most-recent end; miss evicts the least-recent end.
- **FIFO**: hit changes nothing; miss evicts the oldest *loaded* page.
- **Clock**: one reference bit per frame, a hand. Hit: set the bit. Miss: advance the hand, clearing bits that are 1, until a frame with bit 0: evict it, load the new page (bit 0), move the hand on.
- **PostgreSQL clock-sweep**: the bit is a `usage_count` 0–5, +1 per hit, −1 per hand pass, evict at 0; pinned frames are skipped.

Practice

Given. 3 frames, all empty; reference string 1 2 3 1 4 1 2 5 1 2 3 4.

Your task. Misses under LRU, FIFO and clock, with the frame contents after each request for LRU and FIFO.

Solution

request	1	2	3	1	4	1	2	5	1	2	3	4
LRU (old → new)	1	12	123	231	314	341	412	125	251	512	123	234
hit / miss	M	M	M	H	M	H	M	M	H	H	M	M
FIFO (old → new)	1	12	123	123	234	341	412	125	125	125	253	534
hit / miss	M	M	M	H	M	M	M	M	H	H	M	M

LRU: 8 misses. FIFO: 9 misses (page 1 is evicted at the fifth request although it was just used; LRU kept it). Clock: 8 misses, same as LRU here: after 1 2 3 the hit on 1 sets its bit; request 4 finds the hand on frame 1 (page 1, bit 1): clears it and moves on, evicts page 2; and so on. Clock behaves like LRU at a fraction of the bookkeeping, which is the point.

Gotcha

Moving a page on a hit under FIFO. Evicting the least-recently *loaded* under LRU. Forgetting that a clock miss first clears bits that are set, and that the hand moves past the frame it just filled.

17 Sequential flooding, rings, sizing the pool

Method

1. A scan larger than the pool is use-once: under LRU, clock or FIFO it streams through every frame and evicts the whole working set W; the scan itself gains nothing from caching. Reload cost $= W \times T_{\text{miss}}$.
2. PostgreSQL: frames = `shared_buffers`/8 KB; a Seq Scan of a table larger than frames/4 runs through a 32-frame ring (256 KB); W survives iff $W \leq$ frames $- 32$. Below the threshold the table simply becomes resident after one scan. COPY uses a 16 MB ring, VACUUM 256 KB; index scans never ring.
3. Sizing: `shared_buffers` $\approx 25\,\%$ of RAM; size for W (measure it with `pg_buffercache`), not for the database; a streaming workload has $h \approx 0$ at any size; more pool starves the OS cache and `work_mem`.

Practice

Given. `shared_buffers = 256MB`. Working set 12,000 pages, hit ratio 99.5 % by day. Every night a job scans `orders` (28,038 pages); every morning the first minutes are slow. A second job scans `customers` (2,500 pages).

Your task. (a) Under plain LRU: pages of W that survive the `orders` scan and the morning reload cost on SSD. (b) PostgreSQL: frames, threshold, is the `orders` scan ringed, does W survive, what does the ring cost the scan? (c) Same for the `customers` scan. (d) The DBA wants `shared_buffers = 2GB` "so the scan fits": two sentences.

Solution

(a) Frames = 256 MB/8 KB = 32,768. The scan is 28,038 pages, less than the pool, so under LRU it evicts 28,038 of the 32,768 least recent pages: at most $32{,}768 - 28{,}038 = 4{,}730$ of the 12,000 hot pages survive, and only if they were the most recently used; the typical morning reloads about 7,300–12,000 pages at 100 μs: 0.7–1.2 s of misses, spread over the first transactions.

(b) Threshold $= 32{,}768/4 = 8{,}192$ pages; $28{,}038 > 8{,}192$: ringed. The scan recycles 32 frames; $12{,}000 \leq 32{,}736$: W survives intact. The ring costs the scan nothing: every scan page is a compulsory miss that would not be reused. So on PostgreSQL the slow morning comes from somewhere else (the OS cache, which the ring does not protect; a COPY ring; a checkpoint): check `pg_buffercache` after the job.

(c) $2{,}500 \leq 8{,}192$: not ringed; `customers` goes through the pool and stays resident, which is exactly what the dashboard of 15 needed.

(d) The scan gains nothing from 2 GB: a use-once read has hit ratio 0 at any size and is ringed anyway. Measure W first: 12,000 pages = 94 MB already fit in 256 MB; the morning problem is not pool size, and 2 GB would starve the OS cache that serves the `reads`.

Gotcha

"LRU always beats FIFO" (not on a scan). Comparing the table with `shared_buffers` instead of a quarter of it. Thinking the ring slows the scan. "More cache is always better".

18 Steal, no-force, WAL: what the log must hold

Method

Steal: a dirty page of an uncommitted transaction may be written to disk (the pool needs the frame) ⇒ the log needs *before-images* to **undo**. **No-force**: commit does not write the data pages ⇒ the log needs *after-images* to **redo**. **WAL rule**: a log record reaches disk before the data page it describes; at commit the log is forced (one sequential append); data pages go later. Recovery: redo every committed change, undo every uncommitted one. Committed data is safe in the log, not necessarily in the data file.

Practice

Given. T_1 updates page P and commits; T_2 updates page Q, which the pool writes to disk to free a frame; the server crashes before P is written and before T_2 commits.

Your task. (a) State of P and Q on disk at the crash. (b) What the log must contain, and why the policy pair is steal + no-force. (c) What recovery does to P and Q.

Solution

(a) P on disk is *old* (the commit did not write it: no-force). Q on disk is *new*, holding an uncommitted change (steal).

(b) For P: T_1's after-image, written and forced before the commit record, so the commit is durable without writing P. For Q: T_2's before-image, written before Q was flushed (write-ahead), so the premature write can be reversed. Steal keeps the pool free to evict anything; no-force makes commit a single sequential log write instead of random page writes.

(c) Redo T_1: apply its after-image to P. Undo T_2: restore Q from its before-image, then write an abort record. The user who got "committed" for T_1 loses nothing; T_2 vanishes as if never run.

Gotcha

"No-force loses committed data" (the log has it). "Commit writes the pages" (it writes the log). Confusing which image (before/after) serves undo and redo.

Exam technique: tickets and quick checks

19 The ticket method, with three worked tickets

Method

Half a page, four moves, in this order: **claim** (right, wrong, or "both") → **mechanism** (which subsystem, which rule) → **one number** with units, from this course's formulas → **consequence**: what to do instead, and the tempting wrong answer. A ticket answered without a number scores half.

Practice

Given. Three tickets. (1) "The returns report is slow, add an index on `status`." (2) "Set `work_mem = 2GB` in `postgresql.conf`, the nightly sort spills." (3) "Our hit ratio is 98 %, the buffer pool is not the problem."

Your task. Answer each in the four moves.

Solution

(1) Index on status. **Claim**: wrong as stated, right with a different index. **Mechanism**: an index costs the heap pages it touches; `status = 'returned'` keeps 5 % of rows with no physical order. **Number**: $28{,}038 \times (1 - 0.95^{107}) = 27{,}922$ pages, 99.6 % of the table, so a plain index scan costs 114,379 units against 65,538 for the Seq Scan, and a bitmap scan reads the same pages. **Consequence**: do not create it; create the covering index `(status) INCLUDE (customer_id, amount)` (429 leaf pages, no heap) or the partial one if the report is only about returns; and check first whether the report's time is in the scan at all (in the dashboard it was 12 %; the join owned the rest). Tempting answer: "5 % is selective".

(2) work_mem = 2GB. **Claim**: wrong place, right idea. **Mechanism**: `work_mem` is granted *per sort or hash node, per connection*; a server-wide 2 GB lets one report with three sorts take 6 GB and 100 connections exhaust RAM, and the OS cache (which serves the `reads`) is what shrinks. **Number**: the nightly sort of 28,038 pages needs $M \geq 168$ pages = 1.3 MB for one merge pass and 219 MB to avoid merging altogether; the `Disk:` figure of its `Sort Method` line is the exact amount. **Consequence**: `BEGIN; SET LOCAL work_mem = '256MB'; <the sort>; COMMIT;` in the job, default unchanged elsewhere. Tempting answer: "more memory is harmless".

(3) 98 % hit ratio. **Claim**: wrong. **Mechanism**: EAT $= hT_{\text{hit}} + (1-h)T_{\text{miss}}$, the miss term dominates. **Number**: $0.98 \times 0.1 + 0.02 \times 100 = 2.1\,\mu$s, 21 times RAM; at 99.9 % it is $0.2\,\mu$s. And the ratio is a global average: the dashboard's 90 % hid 50,000 random reads on one node, 3 s. **Consequence**: measure the working set with `pg_buffercache` and the `reads` per node with `EXPLAIN (ANALYZE, BUFFERS)`; size `shared_buffers` for W (here 94 MB), make `customers` resident, and aim at 99.9 %. Tempting answer: "98 is an A".

Gotcha

Agreeing or refusing without the mechanism. No number. Not saying what to do instead. Numbers without units.

20 Quick checks: the letter and the one line

Method

One point for the letter, two for the reason: a number, a formula or one mechanism sentence. Write the reason as "because ⟨rule⟩, so ⟨number⟩". The reason also rules out the neighbouring options.

Practice

Given. (a) Which node cannot emit before reading all its input: Seq Scan with Filter / Nested Loop / Sort / Merge Join? (b) `WHERE lower(email) = ?`: B-tree on `email` / B-tree on `lower(email)` / hash on `email` / BRIN on `email`? (c) Rows for `status = 'returned' AND country = 'FR'`: 30,000 / 150,000 / 600,000 / 750,000? (d) Clock-sweep hand reaches an unpinned frame with `usage_count` 2: evicted / becomes 1 and the hand moves on / becomes 3 / pinned? (e) After deleting 40 % of `orders` at random and `VACUUM`: 28,038 / 16,823 / 11,215 / 0 pages? (f) Sort of 28,038 pages with $M = 200$, total I/O: 28,038 / 56,076 / 112,152 / 168,228?

Your task. Letter and one line each.

Solution

(a) **Sort**: the smallest key is known only after the last input row; the others emit as they go (Merge Join needs sorted *inputs*, the sorting is a separate node). (b) **B-tree on `lower(email)`**: an index serves its own key expression; the others index a different key, BRIN also needs physical order. (c) **30,000**: $0.05 \times 0.20 \times 3{,}000{,}000$; 150,000 and 600,000 are the single predicates, 750,000 their sum. (d) **Becomes 1, hand moves on**: the hand decrements and evicts only at 0; hits increment. (e) **28,038**: VACUUM reuses space inside pages and truncates only trailing empty pages; 16,823 is VACUUM FULL. (f) **112,152**: 141 runs ≤ 199, one merge pass, $2 \times 28{,}038 \times 2$; 168,228 is two merge passes, 56,076 one pass counted once.

Gotcha

A letter alone. A reason that does not contain the rule or the number. Picking the option that matches the *trap* computation (sum instead of product, reads only, M instead of $M - 1$).